PYTHON · OBJECT-ORIENTED PROGRAMMING

Think in objects, not just instructions.

A complete, hands-on tutorial that takes you from your first `class` keyword to designing multi-class systems with inheritance, polymorphism, and encapsulation — the way real Python codebases are built.

⏱ ~75 minutes | 🎯 4 modules · 16 quiz questions | 🏆 XP + Mastery Badges | 🧩 Final coding challenge

LEARNING OBJECTIVES

Design classes, control access with encapsulation, build inheritance hierarchies, and write polymorphic code that works across object types.

PREREQUISITES

Comfortable writing Python functions, variables, lists/dicts, and basic control flow (if/for/while).

EXPECTED OUTCOMES

You'll design a small object-oriented system from scratch and explain *why* OOP tools exist, not just their syntax.

ESTIMATED COMPLETION

4 modules × ~15–20 min, self-paced, quizzes gate progress.

🏅 **Reward System:** Earn XP for every quiz question, unlock a Mastery Badge after each module (Novice → Practitioner → Architect → Master), and collect a completion certificate score at the end. Your progress bar above fills as you go — try to finish above 90% average!

MODULE 1 · FOUNDATIONAL

Classes, Objects & Attributes

The blueprint-vs-thing distinction that everything else in OOP builds on.

Every Python value you've used — strings, lists, even integers — is an **object**: a bundle of data plus the behavior that acts on that data. Object-oriented programming just lets *you* design your own kinds of objects instead of only using Python's built-in ones.

A **class** is the blueprint. An **object** (or **instance**) is a specific thing built from that blueprint. You can make a hundred cars from one blueprint — each car is separate, but they all share the same design.

ANALOGY

Think of a `class` as a cookie cutter and each `object` as a cookie. The cutter defines the shape (what attributes and methods every cookie will have), but each cookie you stamp out is its own physical cookie — decorate one with sprinkles and the others are untouched. Changing the cutter's shape (editing the class) changes all *future* cookies, but not cookies you've already baked.

Anatomy of a class

```
class Dog:
  def __init__(self, name, breed):  # the constructor
    self.name = name     # instance attribute
    self.breed = breed

  def bark(self):  # instance method
    return f"{self.name} says Woof!"
```

```
class Dog:
    # __init__ runs automatically when you create an object
    def __init__(self, name, breed):
        self.name = name        # attribute: data
        self.breed = breed

    def bark(self):     # method: behavior
        return f"{self.name} says Woof!"
```

```
# --- creating objects (instances) ---
rex = Dog("Rex", "Labrador")
milo = Dog("Milo", "Pug")

print(rex.bark())   # Rex says Woof!
print(milo.name)  # Milo
```

self — why every method needs it

`self` is how a method refers to *the specific object it was called on*. When you write `rex.bark()`, Python actually calls `Dog.bark(rex)` behind the scenes — `rex` gets passed in as `self` automatically. That's why `self.name` inside `bark()` correctly returns "Rex" and not "Milo".

Class attributes vs instance attributes

INSTANCE ATTRIBUTE	CLASS ATTRIBUTE
Defined via `self.x = ...` inside methods, usually `__init__`	Defined directly inside the class body, outside any method
Unique per object — `rex.name ≠ milo.name`	Shared by every object of that class unless overridden
Example: `self.name`	Example: `species = "Canine"` shared by all dogs

```
class Dog:
    species = "Canis familiaris"   # class attribute — shared

    def __init__(self, name):
        self.name = name              # instance attribute — unique

rex = Dog("Rex")
milo = Dog("Milo")
print(rex.species is milo.species)  # True — same object in memory
```

🔧 Try it: build a Dog object

Name

Buddy

```
Breed
```

```
Beagle
```

```
>>> rex = Dog("Buddy", "Beagle")
>>> rex.bark()
'Buddy says Woof!'
>>> rex.name, rex.breed
('Buddy', 'Beagle')
```

Exercise

Write a `Book` class with instance attributes `title`, `author`, and `pages`, plus a method `summary()` that returns a string like `"Dune by Frank Herbert (412 pages)"`.

```python
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

    def summary(self):
        return f"{self.title} by {self.author} ({self.pages} pages)"
```

Common Misconception

"A class attribute is just a default value copied into each object." It isn't copied — it's shared. If the class attribute is mutable (like a list) and one object mutates it in place, *every* object sees the change. This is a classic source of Python bugs: `self.items.append(x)` on a class-level list affects all instances.

Key Takeaways

- A class is a blueprint; an object is a specific instance built from it.
- `__init__` runs automatically on creation to set up instance attributes.
- `self` refers to "the object this method was called on."

- Instance attributes are unique per object; class attributes are shared.

Module 1 Checkpoint

4 questions · answer all, then submit

1. What is the relationship between a class and an object?

A class is one specific object

An object is an instance created from a class blueprint

They are exactly the same thing

A class can only ever create one object

✅ Correct. A class defines the structure/behavior; an object is a concrete instance built from that blueprint. You can create many objects from one class.

---

2. What does `self` refer to inside a method?

The class itself, not any object

A global variable

The specific object the method was called on

Nothing — it's optional syntax

✅ Correct. self is automatically bound to the instance the method was called on, e.g. rex.bark() passes rex as self.

---

3. Which best describes a class attribute?

Unique to each instance

Shared across all instances unless overridden

Only usable inside __init__

Automatically private

✅ Correct. Class attributes live on the class itself and are shared by every instance unless an instance sets its own attribute of the same name.

---

4. Why is __init__ useful?

It deletes objects when done

It runs automatically to set up a new object's initial state

It's required for all Python files

It replaces the need for methods

✅ Correct. __init__ is the constructor — it runs automatically when an object is created, typically to assign initial instance attributes.

4/4 (100%)

🌟 Perfect! You have fully mastered this module.

Unlock Module 2 →

MODULE 2 · BUILDING ON THE BASICS

Encapsulation & Data Protection

Controlling how an object's internal state can be read and changed.

Encapsulation means bundling data with the methods that operate on it, and restricting direct outside access to that data so it can only change in valid ways. In Python, encapsulation is achieved through convention and naming, not hard enforcement.

ANALOGY

A vending machine encapsulates its cash box. You interact through a public interface — insert coins, press a button — and the machine decides internally how to validate the transaction and dispense the item. You never reach in and grab the cash box directly (well-designed vending machine, well-designed class). The machine's *interface* is public; its *internals* are protected.

Naming conventions for access control

CONVENTION	SYNTAX	MEANING
Public	`self.name`	Freely accessible from anywhere
Protected	`self._name`	"Internal use" — a convention, not enforced
Private	`self.__name`	Name-mangled to `_ClassName__name`, discourages accidental external access

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
```

```
    def get_balance(self):
        return self.__balance

acc = BankAccount(100)
acc.deposit(50)
print(acc.get_balance())    # 150
# acc.__balance  -> AttributeError (name-mangled)
```

The Pythonic way: @property

Instead of writing manual `get_x()`/`set_x()` methods like in Java, Python lets you expose a controlled attribute-style interface with `@property`, so validation runs invisibly whenever code does `acc.balance = ...`.

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("Balance can't be negative")
        self._balance = value

acc = BankAccount(100)
acc.balance = 200      # looks like an attribute, runs validation
# acc.balance = -50  -> raises ValueError
```

🔧 Try it: deposit/withdraw with validation

Balance: ₹100

50

Exercise

Add a `@property` called `pages` to a `Book` class that raises `ValueError` if set to a number less than 1. Sketch the getter and setter.

Common Misconception

"Double underscore means truly private, like a security wall." Python doesn't enforce access control the way Java's `private` does. `__balance` is only *name-mangled* to `_BankAccount__balance` — it's still reachable if someone really wants to. Python trusts convention over enforcement ("we're all consenting adults here").

Key Takeaways

- Encapsulation bundles data + behavior and guards internal state through a public interface.
- `_name` is a convention; `__name` triggers name mangling — neither is truly private.
- `@property` lets validation run behind normal attribute syntax.

Module 2 Checkpoint

4 questions · answer all, then submit

1. In Python, what does a single leading underscore (`_name`) signal?

- Truly private, enforced by the interpreter
- A convention meaning 'internal use', not enforced
- A syntax error
- A class attribute

✅ Correct. A single underscore is purely a convention telling other developers 'treat this as internal' — Python does not block access to it.

2. What does Python do with `self.__balance` (double underscore)?

- Nothing special
- Deletes it after use
- Name-mangles it to _ClassName__balance
- Makes it a class attribute

✅ Correct. Double-underscore attributes are name-mangled to _ClassName__name, which discourages accidental access but doesn't make it impossible.

---

3. What is the main benefit of using @property over a public attribute?

- It's faster at runtime
- It allows validation/logic to run while keeping attribute-style syntax
- It makes the attribute private automatically
- It's required by Python syntax rules

✅ Correct. @property lets you run code (like validation) on get/set while the caller still writes obj.balance = value as if it were a plain attribute.

---

4. Which real-world analogy best fits encapsulation?

- A cookie cutter and cookies
- A vending machine hiding its internal cash box behind a simple interface
- A company org chart
- A universal remote control

✅ Correct. Encapsulation is about exposing a controlled public interface while protecting internal state — like a vending machine's buttons vs its internal cash box.

4/4 (100%)

🌟 Perfect! You have fully mastered this module.

Unlock Module 3 →

MODULE 3 · PRACTICAL APPLICATION

Inheritance & Class Hierarchies

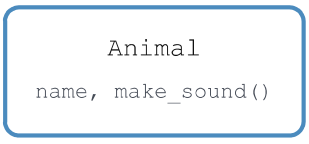
Reusing and specializing behavior across related classes.

Inheritance lets a class (the **subclass** / **child**) reuse and extend the attributes and methods of another class (the **superclass** / **parent**). It models "is-a" relationships: a `Labrador` *is a* `Dog`, a `Dog` *is an* `Animal`.

ANALOGY

Think of a company org chart for job templates. A "Manager" role inherits everything from "Employee" (clock in, get paid) but adds new responsibilities (approve leave). You don't rewrite "clock in" for every role — you inherit it, and override only what's genuinely different.

Basic inheritance




```
class Animal:
    def __init__(self, name):
        self.name = name

    def make_sound(self):
        return "..."

class Dog(Animal):                # Dog inherits from Animal
    def make_sound(self):          # override
        return f"{self.name} says Woof!"

class Cat(Animal):
    def make_sound(self):
        return f"{self.name} says Meow!"
```

super() — extending, not replacing

`super()` calls the parent class's version of a method so you can add to it instead of rewriting it entirely.

```
class Puppy(Dog):
    def __init__(self, name, age_months):
        super().__init__(name)        # reuse Animal's setup
        self.age_months = age_months

    def make_sound(self):
        original = super().make_sound()   # reuse Dog's sound
        return original + " (but it's more of a yip)"
```

Single vs multiple inheritance

SINGLE INHERITANCE	MULTIPLE INHERITANCE

`class Dog(Animal):` — one parent	`class RoboDog(Dog, Robot):` — several parents
Simple, predictable method resolution	Powerful but resolved via MRO (Method Resolution Order) — can get complex
Most everyday hierarchies	Mixins: small reusable behavior classes

🔧 Try it: polymorphic sound check (see Module 4 for full polymorphism)

```
>>> for a in [Dog("Rex"), Cat("Milo"), Puppy("Fifi", 3)]:
...     print(a.make_sound())
Rex says Woof!
Milo says Meow!
Fifi says Woof! (but it's more of a yip)
```

Exercise

Create a `Vehicle` class with `__init__(self, brand)` and a method `describe()`. Then create an `ElectricCar(Vehicle)` subclass that adds a `battery_kwh` attribute and overrides `describe()` to include it, calling `super().describe()` inside.

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand
    def describe(self):
        return f"{self.brand} vehicle"

class ElectricCar(Vehicle):
    def __init__(self, brand, battery_kwh):
        super().__init__(brand)
        self.battery_kwh = battery_kwh
    def describe(self):
        base = super().describe()
        return f"{base} with a {self.battery_kwh}kWh battery"
```

Common Misconception

"Overriding a method deletes the parent's version." It doesn't — the parent method still exists and can be reached with `super()`. Overriding only changes what gets called *by default* when you call the method on the child instance.

Key Takeaways

- Inheritance models "is-a" relationships and avoids duplicating shared behavior.
- Subclasses can override methods to specialize behavior.
- `super()` lets you extend a parent's method rather than replace it outright.
- Python supports multiple inheritance, resolved through MRO.

Module 3 Checkpoint

4 questions · answer all, then submit

1. What relationship does inheritance model?

- 'has-a'
- 'is-a'
- 'uses-a'
- 'not-a'

✅ Correct. Inheritance models 'is-a' relationships — a Dog is an Animal, a Manager is an Employee.

---

2. What does calling `super().__init__(...)` typically do?

- Deletes the parent class
- Calls the parent class's constructor to reuse its setup logic

Creates a new unrelated class

Is only valid in Java, not Python

✅ Correct. super() gives access to the parent class's methods, letting a subclass reuse and extend rather than duplicate the parent's logic.

---

3. If a subclass overrides a method, what happens to the parent's version?

It's permanently deleted

It still exists and can be called via super()

It automatically runs first every time

It becomes private

✅ Correct. Overriding just changes what's called by default on the subclass — the parent version remains accessible through super().

---

4. What resolves which method runs when multiple parent classes define the same method (multiple inheritance)?

Alphabetical order of class names

Method Resolution Order (MRO)

Whichever class was defined first in the file, always

Python raises an error automatically

✅ Correct. Python uses the C3 linearization algorithm to compute a consistent Method Resolution Order (MRO) for multiple inheritance.

4/4 (100%)

Perfect! You have fully mastered this module.

Unlock Module 4 →

MODULE 4 · MASTERY

Polymorphism, Abstraction & Dunder Methods

Writing code that works across object types, and shaping how your objects behave with built-ins.

Polymorphism ("many forms") means code can call the same method name on different object types and get type-appropriate behavior, without knowing or caring about the exact class. **Abstraction** means defining a required interface (a contract) that subclasses must implement, without providing a usable implementation itself.

ANALOGY

A universal TV remote's "power" button is polymorphic: press it on a Sony, Samsung, or LG TV, and each brand's internal circuitry does something different, but you interact through one identical button. You don't need a different remote per brand — that's the power of a shared interface with different implementations.

Polymorphism in action

```
for animal in [Dog("Rex"), Cat("Milo"), Puppy("Fifi", 3)]:
    print(animal.make_sound())   # each class supplies its own behavior

# duck typing: Python doesn't check the type, only that the method exists
class Duck:
    def make_sound(self): return "Quack!"
```

```
# Duck() also works in the loop above — it "quacks like" the interface expects
```

Abstract base classes

Use Python's `abc` module to force subclasses to implement specific methods, catching missing implementations early instead of failing mysteriously at runtime.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self): ...

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2

# Shape()         -> TypeError: can't instantiate abstract class
# Circle(5).area() -> 78.53975   works fine, area() is implemented
```

Dunder (magic) methods

These let your custom objects work naturally with Python's built-in syntax — printing, comparisons, arithmetic — instead of requiring special-case methods.

METHOD	TRIGGERED BY
`__init__`	`MyClass(...)`
`__str__`	`print(obj)` / `str(obj)`
`__eq__`	`obj1 == obj2`
`__len__`	`len(obj)`
`__add__`	`obj1 + obj2`

```
class Point:
    def __init__(self, x, y): self.x, self.y = x, y
    def __str__(self): return f"({self.x}, {self.y})"
```

```
    def __add__(self, other): return Point(self.x+other.x, self.y+other.y)

p1, p2 = Point(1,2), Point(3,4)
print(p1 + p2)    # (4, 6) — thanks to __add__ and __str__
```

🔧 Try it: Point addition simulator

Point A (x,y)

1,2

Point B (x,y)

3,4

```
>>> p1 = Point(1, 2)
>>> p2 = Point(3, 4)
>>> print(p1 + p2)
(4, 6)
```

Exercise

Add `__eq__` to the `Point` class so that `Point(1,2) == Point(1,2)` returns `True`. Sketch the method.

```
def __eq__(self, other):
    return self.x == other.x and self.y == other.y
```

Common Misconception

"Polymorphism requires inheritance." It doesn't, in Python. **Duck typing** means any object with the right method works, related by inheritance or not — "if it walks like a duck and quacks like a duck." Python favors flexible behavior-based contracts over strict type hierarchies.

Key Takeaways

- Polymorphism lets identical calling code work across different object types.

- Abstract base classes (`ABC`/`@abstractmethod`) enforce a required interface.
- Dunder methods integrate custom objects with Python's built-in syntax.
- Duck typing means behavior matters more than declared type.

Module 4 Checkpoint

4 questions · answer all, then submit

1. What is polymorphism, in practice?

Having many unrelated classes

Calling the same method name on different object types and getting type-appropriate behavior

Making all attributes private

Deleting unused methods

✅ Correct. Polymorphism means the same method call works across different classes, each supplying its own appropriate implementation.

---

2. What does an abstract method (via @abstractmethod) enforce?

Nothing, it's just documentation

Subclasses must implement it or they cannot be instantiated

It runs automatically in the parent class

It makes a method private

✅ Correct. @abstractmethod on an ABC-based class prevents instantiating any subclass that hasn't provided a concrete implementation.

---

3. What triggers a class's `__str__` method?

- Calling print(obj) or str(obj)
- Calling len(obj)
- Adding two objects
- Comparing objects with ==

✅ Correct. __str__ defines the readable string representation used by print() and str().

---

4. What is 'duck typing'?

- A requirement that classes inherit from a common ancestor
- Caring about whether an object has the right behavior/methods rather than its exact type
- A Python syntax error
- A dunder method

✅ Correct. Duck typing: 'if it walks like a duck and quacks like a duck' — Python cares about behavior (methods present), not declared inheritance.

4/4 (100%)

🌟 Perfect! You have fully mastered this module.

See Final Challenge →

FINAL STAGE

Practical Challenge & Wrap-Up

Bring together classes, encapsulation, inheritance, and polymorphism in one design.

🏆 Final Practical Challenge

Design a small **employee management system**:

- An `Employee` base class with `name`, private `__salary`, and a method `annual_bonus()` returning 5% of salary.
- A `Manager(Employee)` subclass that overrides `annual_bonus()` to return 10%, using `super()` where useful.
- A `__str__` method so `print(employee)` shows `"Name – role"`.
- Loop over a list of mixed `Employee`/`Manager` objects and print each bonus polymorphically.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary  # Private attribute

    def get_salary(self):
        return self.__salary

    def annual_bonus(self):
        return self.__salary * 0.05  # 5% bonus
```

Check my approach

- ✅ Defines a class
- ✅ Uses inheritance (Manager(Employee))
- ✅ Uses super()
- ✅ Defines __str__
- ✅ Encapsulates salary (e.g. __salary)

This is a self-check heuristic, not a real interpreter — compare your structure against the cheat sheet above.

Cheat Sheet

Define & instantiate

```
class C:
    def __init__(self, x):
        self.x = x
obj = C(1)
```

Inheritance

```
class Sub(Base):
    def m(self):
        return super().m()
```

Encapsulation

```
self._protected
self.__private
@property
def x(self): return self._x
```

Abstract class

```
from abc import ABC, abstractmethod
class A(ABC):
    @abstractmethod
    def m(self): ...
```

Common dunders

```
__init__ __str__ __eq__
__len__ __add__ __repr__
```

Polymorphism / duck typing

```
for o in objs:
    o.speak()  # any class works
```

Summary Notes

Classes/objects: a class is a blueprint, an object an instance; `self` ties a method to its instance.
Encapsulation: bundle data + behavior; use _/__ conventions and `@property` for controlled access.
Inheritance: reuse and specialize behavior across an "is-a" hierarchy using `super()`.
Polymorphism/Abstraction: write code against a shared interface; use `ABC` to enforce required methods; dunder methods integrate with Python syntax.

Continue Learning

📚 BOOKS

- "Fluent Python" — Luciano Ramalho
- "Python Object-Oriented Programming" — Steven Lott & Dusty Phillips
- "Clean Code" — Robert C. Martin (OOP design principles)

📖 DOCUMENTATION

- docs.python.org — Classes tutorial
- docs.python.org — abc module reference
- docs.python.org — Data model (dunder methods)

📄 RESEARCH & WRITING

- Liskov, B. "Data Abstraction and Hierarchy" (1987)
- Cook, W. "On Understanding Data Abstraction, Revisited"

👥 COMMUNITIES

- r/learnpython, r/Python
- Python Discord
- Real Python community forum

💻 PRACTICE PLATFORMS

- Exercism.org — Python track (OOP exercises)
- LeetCode / HackerRank — design-pattern problems
- CodeWars — kata katas tagged "OOP"

🔍 SEARCH KEYWORDS

`python dunder methods`

`python property decorator`

`python abc abstractmethod`

`python MRO method resolution order`

`python composition vs inheritance`

Additional AI Prompts For Further Learning

- "Explain composition vs inheritance in Python with a real-world refactoring example."
- "Show me Python's method resolution order (MRO) with a multiple-inheritance diamond example."
- "Walk me through building a small plugin system in Python using ABCs."
- "Compare Python dataclasses to writing a class by hand — when should I use each?"

```
Final score: 16/16 (100%) — Mastery level: OOP Master
```

Interactive Learning Studio · Python OOP · Built for hands-on mastery.